

Lecture 1: Introduction

EC7412 Part II: Data Science for Economists

Adam Altmejd Selder
Swedish Institute for Social Research (SOFI)
adam.altmejd@sofi.su.se

2025-04-09

Introduction

Welcome to a half-course in *Data Science for Economists*!

Who am I?

- Adam Altmejd Selder — adam.altmejd@sofi.su.se
- I study inequality, education, health at SOFI
- But really... I spend 90% of my time matching, merging, filtering, sorting, cleaning, and visualizing data

Who are you?

Tell me your name, interests, coding background, what you hope to get out of this (half)-course.

Introduction

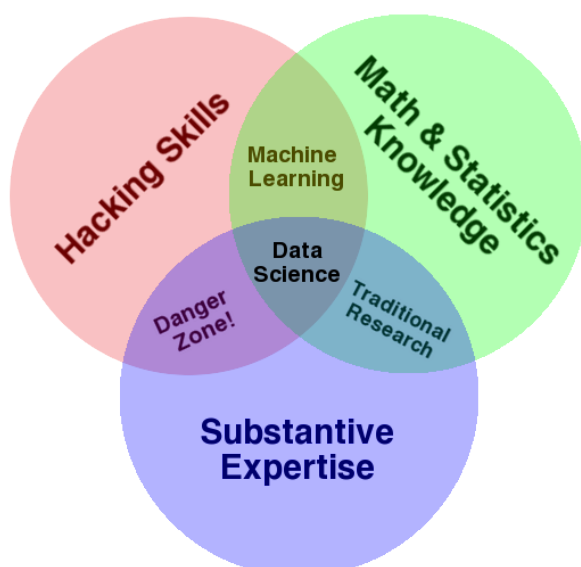


Figure 1:

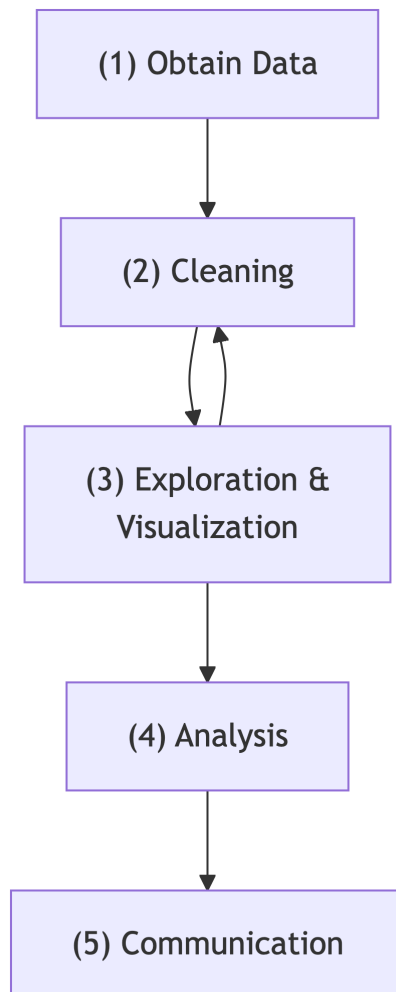
- This course: adding “hacking skills” to your skill set
- Learn *efficient* data collection, wrangling, visualization

- Tools to make your work as economists more productive!

HT: Drew Conway

Hacking skills (efficient file manipulation, shell, algorithmic/programming mindset), Statistics (to extract insights), Substantive expertise (building knowledge, theory). Danger zone? “lies, damn lies, and statistics” - being able to ask questions and process data, but not interpreting analyses.

The empirical research process



- Most Economics education focuses on step 1 and 4 (and maybe 5).
- This course will be about step 2 and 3.

Why should you learn data science?

- Economists now crunch more data than ever!
 - Financial data, sales/marketing data, insurance data, health data,
 - Public sector data (especially in Sweden!)

Why this course?

- An econ master already gives you domain and stats knowledge.

- Nothing on how to actually do 90% of the work efficiently
- I use these tools daily ... but I have not run a DSGE model since 2013.
- The data science toolkit gives you *complementary* skills
- Will make you more productive, and more *employable*

R for data science

- We will mostly be working in R
- Free, open source
- Second most popular data science language after Python
- Good metrics support, integrates well with Stata
- It's the language I use most days
- Actually quite similar to Python and Julia (which I also recommend!)

Why not Stata?

- Not a programming language
- Does not support the functionality or “mindset” I want you to learn
- ... but Stata is great for the “analysis” step of the pipeline!
- Purpose of this course is to teach you to use the **right tool for the job**, can be Stata for some things

Stata vs. R

- For data wrangling and visualization, R is superior:
 - Way larger feature set
 - Much faster (often 5-10 times as fast!)
 - Parallelized processing for free
- No private sector data scientist uses Stata
- Easier to get help: both on Stack Overflow and from LLMs

See this benchmark for speed differences: <https://github.com/matthieugomez/benchmark-stata-r>

Why coding?

- You can get quite far clicking through menus in Stata and using Spreadsheets
- Code is reusable and replicable
- “Emulated” by Stata .do-files:
 - Collections of commands
 - Different from programming: less flexible, lacks many features
- Still, some things are better done in Excel (or Stata) — always ask yourself what the best and simplest tool for the job is!

Data wrangling intro

Common data problems

- Invalid/inconsistent values
- Improperly formatted missing values
- Messy naming
- Duplicates
- Mixed-up date formats

- Missing or non-unique keys
- Encoding

https://vdsbook.com/04-data_cleaning#sec-examine-clean

Same data, different structure

```
library(tidyverse)
table2
```

```
# A tibble: 12 × 4
  country    year type      count
  <chr>    <dbl> <chr>    <dbl>
1 Afghanistan 1999 cases      745
2 Afghanistan 1999 population 19987071
3 Afghanistan 2000 cases     2666
4 Afghanistan 2000 population 20595360
5 Brazil      1999 cases     37737
6 Brazil      1999 population 172006362
7 Brazil      2000 cases     80488
8 Brazil      2000 population 174504898
9 China       1999 cases     212258
10 China      1999 population 1272915272
11 China      2000 cases     213766
12 China      2000 population 1280428583
```

```
table3
```

```
# A tibble: 6 × 3
  country    year rate
  <chr>    <dbl> <chr>
1 Afghanistan 1999 745/19987071
2 Afghanistan 2000 2666/20595360
3 Brazil      1999 37737/172006362
4 Brazil      2000 80488/174504898
5 China       1999 212258/1272915272
6 China      2000 213766/1280428583
```

“Tidy” data

- Each **variable** is a column; each column is a variable.
- Each **observation** is a row; each row is an observation.
- Each **value** is a cell; each cell is a single value.

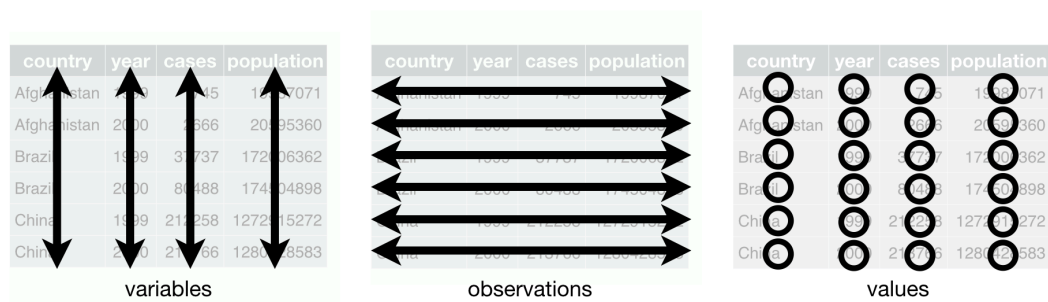


Figure 2:

Making data tidy

Tidying data makes it easier to work with. We will be doing this a lot.

```
library(data.table)
dcast(as.data.table(table2),
      country + year ~ type)
```

```
Key: <country, year>
   country year cases population
   <char> <num> <num>    <num>
1: Afghanistan 1999    745 19987071
2: Afghanistan 2000   2666 20595360
3:      Brazil 1999  37737 172006362
4:      Brazil 2000  80488 174504898
5:        China 1999 212258 1272915272
6:        China 2000 213766 1280428583
```

The R4DS book has a good chapter on tidy data.

Programming principles

Programming principles

- Write **modular** code, in generalized functions
- Don't write the same code twice
- Use a **consistent** coding style and naming scheme
- **Document** everything,
- but write **verbose** code that can be understood in itself
- Test your code

Modular code

- Functional programming: aim to divide code by feature into functions
- Example from my work: collapsing education achievement data
- Why?
 - Reusability
 - Easier to understand
 - Easier to debug and test

No repetition

- Don't repeat the same code
- Instead, use functions, loops, or vectorized calls
- A smaller codebase is easier to understand and debug

Be consistent

- Use the same coding style and naming convention
- For example: don't mix `snakeCase` variable names with `underscore_based` names,
- Use the same indentation (2 or 4 spaces)

Document everything

- Always include a README-file that explains your project
- Use comments to document what your code is doing
- Start functions with a comment explaining the function and its arguments

Be verbose

- By writing verbose code, you will not need as many comments:
- Call a variable `high_school_gpa`, not `hsgpa`, (or worse `question12`)
- Same applies to data labels:
 - `edu_level = primary, secondary, high school` not `1,2,3`
 - `female = TRUE, FALSE` not `Sex = 1,21`

Test your code

- By running tests you can make sure your code does what it is supposed to, even after you change things
- If we have time (and you want to), we can spend some time talking about unit tests and automation towards the end of the course
- We will get back to these principles many times!

Our working environment = VS Code

VS Code IDE

- IDE = integrated development environment
- A **text editor** with tons of extra features:
 - Git and terminal integration
 - AI programming help
 - Integrated remote development system
 - Huge library of extensions

Other great alternatives are RStudio, the newer data science-focused editor Positron, the AI-focused editor Cursor, or the new (and promising) IDE Zed. Out of these, Positron and Cursor build directly on VS Code.

VS Code intro

Let's switch over to VS code and I'll show you some useful things:

- Browsing folders

¹SCB actually does this still!

- Multi-line editing
- Advanced (multi-file) find (and replace)
- Ctrl/Command+Shift+P for accessing all commands
- Ctrl/Command+Shift+P for navigation

AI assistants in VS Code

- GitHub Copilot is integrated into VS Code
- Available for free for students with GitHub education
- Chat, “edits”, inline suggestions

VS Code and R

- To get the most out of coding in R in VS Code we need to install the R extensions:
 - vscode-R
 - R debugger (extra)
 - And the R packages they require

```
install.packages(c("languageserver", "httpgd", "vscDebugger"))
```

- See Problem set 0 for instructions!

Let's do some data science!

Example 1: Gapminder data

Let's load and summarize the famous Gapminder data in R, provided by the gapminder R package.

Example 2: LLM data processing

Let's build a script that calls the OpenAI API to process some survey feedback I just collected.

Concluding remarks

Course Preview

1. Introduction
2. Project organization, version control & the shell
3. Basic R coding
4. Visualization
5. Data wrangling in R
6. Servers, VMs, APIs, & LLMs

Problem sets

Problem set 0 - Setting up data science computer environment - Due before next lecture

Problem set 1 - Covers lectures 2-4

Problem set 2 - Covers lectures 5-6

Problem Set 0: Getting Set Up

- **Install** R, Git, an VS Code
- **Create** a GitHub account, register for student benefits, and configure Copilot

- **Install** VS Code extensions and R packages
- **Test-run** a script to double-check that everything is working

Pep talk

- Frustration is normal
- Coding is hard, even pros get stuck
- Take breaks, breathe, and get help (more on how next time!)
- Debugging errors is how we learn

Concluding remarks

- Learning data science will give you a really useful new toolkit,
- and demystify popular software development techniques
- Please let me know when I'm going too fast or if you want to know more about some subject

Next lecture: Project management, version control, and the shell

Bibliography